Course Outline: Consultando bases de datos con SQL en PostgreSQL (Fundamentos de SQL)

Title: Fundamentos de SQL: Consultando Bases de Datos con PostgreSQL **Learnpack:** + Consultando bases de datos con SQL en PostgreSQL (Fundamentos de SQL) **Prerequisite:** Bases de datos (solo informativo) **Target Audience:** Beginner developers learning database fundamentals and SQL query operations **Total Lessons:** 11 **Format:** Mixed (55% hands-on)

Course Description

This course introduces students to SQL fundamentals through practical PostgreSQL database operations. Students will learn to structure and execute SQL queries, understand relational database concepts, and master CRUD operations with proper filtering and aggregation techniques. The course emphasizes safe query practices, explicit column selection, and handling NULL values effectively.

By the end of this course, students will confidently write SQL queries that retrieve, insert, update, and delete data while understanding the underlying relational database principles that make these operations possible.

Course Philosophy

This course emphasizes:

- **Safe query practices**: Always SELECT before DELETE/UPDATE operations
- **Explicit projection**: Request only needed columns instead of using SELECT *
- **Atomic data design**: One cell equals one value principle
- **Strict typing understanding**: Appreciating database constraints as virtues, not limitations

---

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Database Fundamentals	2
02 - Basic CRUD Operations	3
03 - Filtering and Organizing Data	2
04 - Data Aggregation and NULL Handling	2
05 - Assessment and Mastery	2
06 - Outro	1
Total	**11**

Section 00: Welcome

00.0 Welcome to SQL and Relational Databases 📖

Learning Objectives:

- Understand what SQL is and why it's essential for modern applications
- Recognize the role of relational databases in data management
- Identify the core problems SQL solves in data storage and retrieval
- Preview the practical skills gained through this course

Content Outline:

- The fundamental problem: How do applications store and retrieve structured data?
- What is SQL and why it became the universal database language
- PostgreSQL as our learning environment
- Course journey overview: From basic queries to complex aggregations

Transitions: Next, we'll dive into the foundational concepts of relational tables and data types that make SQL queries possible.

Key Principles:

- SQL is declarative: you specify what you want, not how to get it
- Relational databases enforce structure and consistency through strict rules
- Every SQL operation either succeeds completely or fails completely
- Database design directly impacts query effectiveness

Examples:

- A web application storing user profiles, orders, and products in separate but connected tables
- An e-commerce system needing to find "all orders from last month with total > $100"
- A content management system tracking articles, authors, and categories with relationships

---

Section 01: Database Fundamentals

01.0 What is SQL and How Relational Tables Work 📖

Learning Objectives:

- Explain what SQL is and its primary use cases in modern applications
- Describe how relational tables organize data through rows and columns
- Understand primary keys and their role in table structure
- Differentiate between PostgreSQL data types and their appropriate usage

Content Outline:

- SQL definition and its role as the standard database query language
- Relational table anatomy: tables, rows (records), and columns (fields)
- Primary keys as unique identifiers and why every table needs one
- Auto-incrementing numbers and their practical applications
- PostgreSQL data types: numeric (INTEGER, DECIMAL), text (VARCHAR, TEXT), boolean, and date/time types
- The atomicity principle: one cell, one value

Transitions: Now that we understand the theoretical foundation, let's practice creating tables and inserting data to see these concepts in action.

Key Principles:

- Tables represent entities; rows represent instances; columns represent attributes
- Primary keys ensure each row can be uniquely identified and referenced
- Data types enforce consistency and enable appropriate operations
- Atomicity prevents data corruption and enables reliable queries

Examples:

- A users table with columns: id (auto-increment), name (text), email (text), is_active (boolean), created_at (timestamp)
- Product inventory with: product_id (primary key), name, price (decimal), quantity (integer), in_stock (boolean)
- Blog posts structure: post_id, title, content, author_id, published_date, is_published

---

01.1 Data Types and Primary Keys in Practice 💻

Challenge Prompt: Create a PostgreSQL table for a library book catalog that demonstrates proper use of data types and primary key configuration.

Solution Code:

```
 CREATE TABLE books (
    book_id SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    isbn VARCHAR(13) UNIQUE,
    pages INTEGER,
    price DECIMAL(8,2),
    is_available BOOLEAN DEFAULT true,
    publication_date DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insert sample data to demonstrate data types
INSERT INTO books (title, isbn, pages, price, publication_date) VALUES
('The PostgreSQL Guide', '9781234567890', 350, 29.99, '2023-01-15'),
('SQL Fundamentals', '9780987654321', 280, 24.50, '2022-08-20'),
('Database Design Patterns', '9781122334455', 420, 39.99, '2023-03-10');

SELECT * FROM books;
```

Challenge Code:

```
 -- Create a table for library book catalog
-- Include: book_id (auto-increment primary key), title, isbn, pages, price, availability, publication_date, crea
CREATE TABLE books (
    -- Your table definition here
);

-- Insert at least 2 sample books with different data types
-- Your INSERT statements here

-- Display all books to verify the structure
-- Your SELECT statement here
```

Section 02: Basic CRUD Operations

02.0 Understanding SELECT, INSERT, UPDATE, DELETE 📖

Learning Objectives:

- Define CRUD operations and their correspondence to SQL statements
- Explain why explicit column selection is preferred over SELECT *
- Understand the difference between hard delete and soft delete approaches
- Recognize when each CRUD operation is appropriate in real applications

Content Outline:

- CRUD acronym breakdown: Create (INSERT), Read (SELECT), Update (UPDATE), Delete (DELETE)
- SELECT fundamentals: retrieving data with explicit column specification
- Why SELECT * is problematic: performance, security, and maintenance issues
- INSERT operations: adding new records with proper data types
- UPDATE operations: modifying existing records safely
- DELETE vs soft delete: when to remove vs mark as inactive
- Soft delete implementation using status columns

Transitions: Let's practice these fundamental operations with hands-on SQL queries to solidify your understanding.

Key Principles:

- Explicit projection: always specify needed columns instead of using SELECT *
- Safe mode practice: SELECT before UPDATE/DELETE to verify target rows
- Soft deletes preserve data integrity and enable recovery
- Each CRUD operation should have a clear business purpose

Examples:

- E-commerce: SELECT product_name, price FROM products WHERE is_active = true
- User management: UPDATE users SET last_login = NOW() WHERE user_id = 123
- Content moderation: UPDATE comments SET is_deleted = true WHERE comment_id = 456 (soft delete)

02.1 Basic CRUD Queries 💻

Challenge Prompt: Write SQL queries that perform all four CRUD operations on a products table, demonstrating explicit column selection and safe query practices.

Solution Code:

```
 -- READ: Select specific columns (explicit projection)
SELECT product_name, price, category_id FROM products WHERE is_active = true;

-- CREATE: Insert new product
INSERT INTO products (product_name, price, category_id, is_active)
VALUES ('Wireless Headphones', 79.99, 1, true);

-- UPDATE: Modify existing product (safe mode: WHERE with specific condition)
UPDATE products
SET price = 69.99, updated_at = CURRENT_TIMESTAMP
WHERE product_id = 1 AND product_name = 'Wireless Headphones';

-- DELETE: Remove product (with safety condition)
DELETE FROM products
WHERE product_id = 999 AND is_active = false;

-- Verify changes
SELECT product_id, product_name, price FROM products WHERE product_id <= 5;
```

Challenge Code:

```
 -- Assume products table exists with: product_id, product_name, price, category_id, is_active, created_at, updat

-- READ: Write a SELECT for product names and prices of active products only
-- Your SELECT statement here
```

```
-- CREATE: Insert a new product of your choice
-- Your INSERT statement here

-- UPDATE: Change the price of the product you just inserted
-- Your UPDATE statement here

-- DELETE: Remove a product with product_id = 999 (if it exists and is inactive)
-- Your DELETE statement here
```

02.2 Advanced CRUD with Soft Deletes 💻

Challenge Prompt: Implement a soft delete system for a users table that marks users as deleted instead of removing them permanently.

Solution Code:

```
 -- Soft delete: Mark user as deleted instead of removing
UPDATE users
SET is_deleted = true, deleted_at = CURRENT_TIMESTAMP
WHERE user_id = 5;

-- Query active users only (excluding soft-deleted)
SELECT user_id, username, email
FROM users
WHERE is_deleted = false OR is_deleted IS NULL;

-- Count active vs deleted users
SELECT
    COUNT(*) as total_users,
    COUNT(CASE WHEN is_deleted = true THEN 1 END) as deleted_users,
    COUNT(CASE WHEN is_deleted = false OR is_deleted IS NULL THEN 1 END) as active_users
FROM users;

-- Restore a soft-deleted user
UPDATE users
SET is_deleted = false, deleted_at = NULL
WHERE user_id = 5 AND is_deleted = true;
```

Challenge Code:

```
 -- Assume users table with: user_id, username, email, is_deleted, deleted_at

-- Soft delete user with user_id = 10
-- Your UPDATE statement here

-- Select all active users (not soft-deleted)
-- Your SELECT statement here

-- Count total active and deleted users
-- Your SELECT with COUNT statement here
```

Section 03: Filtering and Organizing Data

03.0 WHERE Clauses and Operators 📖

Learning Objectives:

- Construct WHERE clauses using mathematical and logical operators
- Apply range operators (IN, BETWEEN) for efficient data filtering
- Use text pattern matching with LIKE and ILIKE operators
- Combine multiple conditions with AND, OR, and NOT logic

Content Outline:

- WHERE clause purpose: filtering rows based on conditions
- Mathematical operators: =, !=, <, >, <=, >= for numeric and date comparisons
- Logical operators: AND (both conditions true), OR (either condition true), NOT (condition false)
- Range operators: IN (value in list), BETWEEN (value in range) for cleaner queries
- Text pattern matching: LIKE (case-sensitive), ILIKE (case-insensitive) with wildcards
- Operator precedence and parentheses for complex conditions
- Performance considerations: indexed columns filter faster

Transitions: Now let's practice building complex filtering queries that combine these operators effectively.

Key Principles:

- Filtering reduces data transfer and improves query performance
- Logical operators enable sophisticated condition combinations
- Pattern matching provides flexible text search capabilities
- Explicit conditions are more maintainable than complex OR chains

Examples:

- Customer search: WHERE age BETWEEN 25 AND 45 AND city IN ('Madrid', 'Barcelona')
- Product catalog: WHERE product_name ILIKE '%laptop%' AND price < 1000
- Order history: WHERE order_date >= '2023-01-01' AND (status = 'completed' OR status = 'shipped')

03.1 Complex Filtering Queries 💻

Challenge Prompt: Write SQL queries that demonstrate sophisticated filtering using mathematical, logical, range, and text pattern operators on an e-commerce products database.

Solution Code:

```
 -- Complex filtering with multiple operator types
SELECT product_id, product_name, price, category
FROM products
WHERE price BETWEEN 50.00 AND 200.00
    AND category IN ('Electronics', 'Books', 'Clothing')
    AND product_name ILIKE '%pro%'
    AND is_active = true;

-- Text pattern matching with different wildcards
SELECT product_name, description
FROM products
WHERE (product_name LIKE 'Smart%' OR description ILIKE '%wireless%')
    AND price > 25.00;

-- Logical operators with date ranges
SELECT order_id, customer_id, total_amount, order_date
FROM orders
WHERE order_date >= '2023-06-01'
    AND order_date < '2023-07-01'
```

```
    AND (total_amount > 100 OR customer_id IN (1, 5, 10))
    AND NOT status = 'cancelled';
```

Challenge Code:

```
 -- Assume products table with: product_id, product_name, price, category, description, is_active
-- Assume orders table with: order_id, customer_id, total_amount, order_date, status

-- Find active products priced between $20-$100 in 'Electronics' or 'Books' categories
-- Your SELECT with BETWEEN and IN here

-- Find products with names starting with 'Smart' or descriptions containing 'wireless'
-- Your SELECT with LIKE/ILIKE here

-- Find June 2023 orders over $50 that are not cancelled
-- Your SELECT with date range and logical operators here
```

Section 04: Data Aggregation and NULL Handling

04.0 GROUP BY, Aggregate Functions, and NULL Concepts 📖

Learning Objectives:

- Explain what NULL means in database contexts and why it's different from empty strings or zero
- Apply aggregate functions (COUNT, SUM, AVG, MIN, MAX) to calculate summary statistics
- Use GROUP BY to organize data into meaningful categories for aggregation
- Understand how NULL values affect aggregate calculations and query results

Content Outline:

- NULL concept: absence of value, not zero or empty string
- Why equality (=) doesn't work with NULL; introducing IS NULL and IS NOT NULL
- How NULL impacts aggregate functions: COUNT ignores NULLs, SUM/AVG skip NULLs
- Aggregate functions overview: COUNT (rows), SUM (totals), AVG (averages), MIN/MAX (extremes), DISTINCT (unique values)
- GROUP BY mechanics: partitioning data into groups for separate calculations
- ORDER BY and LIMIT: organizing and restricting result sets
- HAVING clause: filtering groups after aggregation (conceptual introduction)
- Common aggregation patterns in business applications

Transitions: Let's practice these aggregation techniques with real-world scenarios involving sales data and NULL handling.

Key Principles:

- NULL represents unknown/missing data, not zero or empty values
- Aggregate functions handle NULLs predictably but require awareness
- GROUP BY enables powerful data summarization and reporting
- Ordering and limiting results improves usability and performance

Examples:

- Sales reporting: SELECT category, COUNT(*), AVG(price) FROM products GROUP BY category
- NULL handling: SELECT COUNT(*) as total, COUNT(phone) as with_phone FROM customers
- Top performers: SELECT * FROM salespeople ORDER BY total_sales DESC LIMIT 5

04.1 Aggregation and NULL Handling Queries 💻

Challenge Prompt: Create SQL queries that demonstrate data aggregation, grouping, and proper NULL value handling using a sales database.

Solution Code:

```
 -- Aggregation with GROUP BY and NULL awareness
SELECT
    category,
    COUNT(*) as total_products,
    COUNT(price) as products_with_price,
    AVG(price) as average_price,
    MIN(price) as min_price,
    MAX(price) as max_price
FROM products
GROUP BY category
ORDER BY average_price DESC;

-- NULL handling examples
SELECT
    COUNT(*) as total_customers,
    COUNT(phone) as customers_with_phone,
    COUNT(email) as customers_with_email
FROM customers;

-- Complex aggregation with filtering
SELECT
    EXTRACT(MONTH FROM order_date) as month,
    COUNT(*) as total_orders,
    SUM(total_amount) as revenue,
    AVG(total_amount) as avg_order_value
FROM orders
WHERE order_date >= '2023-01-01' AND status != 'cancelled'
GROUP BY EXTRACT(MONTH FROM order_date)
ORDER BY month;
```

Challenge Code:

```
 -- Assume products table: product_id, product_name, price, category (some prices may be NULL)
-- Assume customers table: customer_id, name, email, phone (email/phone may be NULL)
-- Assume orders table: order_id, customer_id, total_amount, order_date, status

-- Group products by category, show count and average price (handle NULL prices)
-- Your SELECT with GROUP BY here

-- Count total customers vs customers with phone numbers (demonstrate NULL handling)
-- Your SELECT with COUNT here

-- Show monthly sales summary excluding cancelled orders
-- Your SELECT with date functions and GROUP BY here
```

Section 05: Assessment and Mastery

05.0 SQL Best Practices and Safety Patterns 📖

Learning Objectives:

- Apply safe query patterns that prevent data loss and ensure query accuracy
- Implement proper naming conventions for database objects
- Recognize and avoid common SQL anti-patterns that compromise performance or safety
- Understand the importance of data type constraints and atomic design principles

Content Outline:

- Safe mode practice: always SELECT before DELETE/UPDATE operations
- Explicit projection benefits: performance, security, maintainability
- Naming conventions: plural table names, descriptive column names
- Atomic design principle: one cell, one value - avoiding comma-separated lists in single columns
- Strict typing as a feature: embracing constraints rather than fighting them
- Query testing strategies: verifying results before applying changes
- Performance awareness: understanding index usage and query optimization basics
- Common anti-patterns to avoid: SELECT *, unqualified DELETEs, mixing data types

Transitions: Let's assess your mastery of these SQL fundamentals through practical scenarios.

Key Principles:

- Safety first: verify before modifying data
- Explicit is better than implicit in column selection
- Database constraints prevent errors rather than causing them
- Atomic data design enables reliable queries and relationships

Examples:

- Safe update pattern: SELECT * FROM users WHERE id = 5; then UPDATE users SET email = 'new@email.com' WHERE id = 5;
- Good naming: users table with user_id, first_name, email_address columns
- Atomic design: separate user_skills table instead of skills column with comma-separated values

---

05.1 SQL Fundamentals Assessment 🧠

Learning Objectives:

- Demonstrate understanding of SQL query construction and CRUD operations
- Apply proper NULL handling and aggregate function usage
- Identify correct filtering and grouping patterns
- Recognize best practices and anti-patterns in SQL development

Question Format: Multiple Choice

Questions:

1. **What is the main reason to avoid using SELECT * in production queries?**

   - A) It's slower than naming columns explicitly
   - B) It returns unnecessary data and makes queries harder to maintain
   - C) It doesn't work with WHERE clauses
   - D) PostgreSQL doesn't support it

   Correct Answer: B

2. **Which query correctly finds all products priced between $50 and $100 in the 'Electronics' category?**

   - A) `SELECT * FROM products WHERE price > 50 AND price < 100 AND category = 'Electronics'`
   - B) `SELECT * FROM products WHERE price BETWEEN 50 AND 100 AND category = 'Electronics'`
   - C) `SELECT * FROM products WHERE price >= 50 OR price <= 100 AND category = 'Electronics'`
   - D) `SELECT * FROM products WHERE price IN (50, 100) AND category = 'Electronics'`

Correct Answer: B

3. **What happens when you use COUNT(phone_number) on a table where some phone_number values are NULL?**
   - A) The query returns an error
   - B) NULL values are counted as 0
   - C) NULL values are ignored and not counted
   - D) NULL values are counted as 1

   Correct Answer: C

4. **Which approach represents a "soft delete" pattern?**
   - A) `DELETE FROM users WHERE user_id = 5`
   - B) `UPDATE users SET is_deleted = true WHERE user_id = 5`
   - C) `DROP TABLE users`
   - D) `TRUNCATE TABLE users`

   Correct Answer: B

5. **What is the safest practice before running an UPDATE statement?**
   - A) Run a SELECT with the same WHERE clause to verify which rows will be affected
   - B) Always use LIMIT 1
   - C) Run the UPDATE without a WHERE clause first
   - D) Use SELECT * FROM the entire table

   Correct Answer: A

Evaluation Criteria:

- 4-5 correct: Excellent understanding of SQL fundamentals and best practices
- 3 correct: Good grasp with minor gaps in safety practices or NULL handling
- 2 correct: Basic understanding present, review aggregation and filtering concepts
- 0-1 correct: Requires comprehensive review of all course materials

Score Interpretation: This assessment validates your ability to write safe, effective SQL queries and understand relational database principles essential for backend development.

Section 06: Outro

06.0 Your SQL Journey Forward

Content Outline:

- Course recap: You've mastered the fundamental building blocks of SQL - CRUD operations, filtering with WHERE clauses, data aggregation with GROUP BY, and crucial NULL handling patterns
- Connection to bigger picture: These SQL skills form the foundation for backend API development, data analysis, and any application requiring persistent data storage
- Next steps and continued learning: Advanced SQL topics (JOINs for multi-table queries, subqueries, window functions), database design principles, and performance optimization
- Resources for further exploration: PostgreSQL official documentation, SQL practice platforms (LeetCode Database, HackerRank SQL), and database design pattern guides
- Encouragement and celebration: You now possess the core skills to interact with any relational database confidently, a capability that powers countless modern applications

Note: This is conclusion only - no theory, exercises, or assessment.
